LINUTRONIX

The New Kernel CPUID API

Linux Plumbers Conference
December 11-13, 2025

x86 Microconference

Ahmed S. Darwish <darwi@linutronix.de>

The New Kernel CPUID API

Why a new call-site API?

(I) Is the CPUID leaf supported by CPU?

Each CPUID call-site query is usually guarded by a check similar to:

- `If (c->cpuid_level >= 0x01) { … }`
- `If (c->cpuid_level >= 0x05) { … }`
- `If (c->extended_cpuid_level >= 0x80000008) { … }`
- `If (c->extended_cpuid_level >= 0x8000001f) { … }`

Where “c” is the boot or the current CPU's capability structure, and the max level(s) are known by querying `CPUID(0x0)` and `CPUID(0x80000000)`.

(II) Is the CPUID leaf supported by CPU x86 vendor?

Some CPUID leaves are supported only by certain x86 vendors, and the kernel is careful not to query them on other CPUs:

- `CPUID(0x2)/CPUID(0x4)`: Intel, Centaur, Zhaoxin
- `CPUID(0x16)`: Intel
- `CPUID(0x8000001d)`: AMD, Hygon
- `CPUID(0x80860000)=>CPUID(0x80860006)`: Transmeta
- `CPUID(0xc0000000)=>CPUID(0xc0000001)`: Centaur, Zhaoxin

(III) Is the CPUID leaf dependent on other leaves?

Some CPU features represented by certain CPUID leaves and bits:

- `CPUID(0x15) [ART_CPUID_LEAF]`: depends on `!X86_FEATURE_HYPERVISOR`, `!tsc_async_resets global`, `X86_FEATURE_NONSTOP_TSC`, and `X86_FEATURE_TSC_ADJUST`
- Dependency table at `arch/x86/kernel/cpu/cpuid-deps.c`

(IV) Is the CPUID leaf dependent on other system state?

Certain system state affects CPUID leaves visibility:

- Disabling legacy CPU serial Number, CPUID(0x3)
- Clearing legacy Intel MSR_IA32_MISC_ENABLE_LIMIT_CPUID_BIT
- Enabling possibly masked Transmeta's CPUID(0x1).EDX feature bits

(V) Feature bit within CPUID leaf

After tackling all previous issues, the feature bit can then be queried:

- CPUID(0x1):

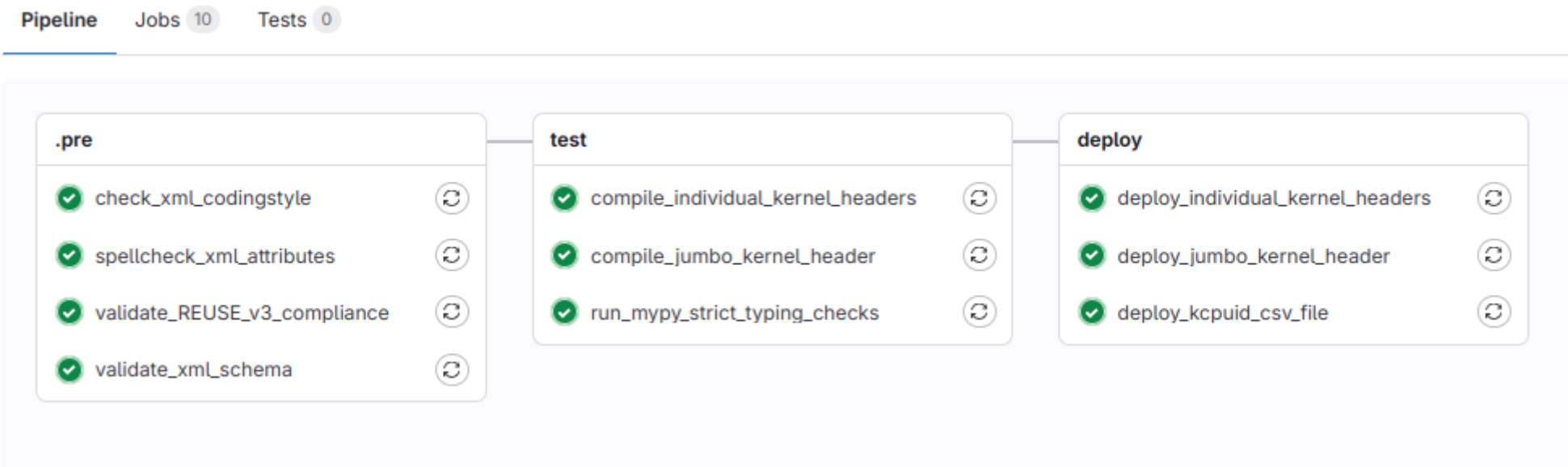
```
cpuid(0x00000001, &tfms, &misc, &junk, &cap0);
if (cap0 & (1<<19)) {
  c->x86_clflush_size = ((misc >> 8) & 0xff) * 8;
}
```

The New Kernel CPUID API

Tackling these issues

(I) The x86-cpuid-db project

Discussed at previous talks



(II) Refactoring old code paths

CPUID(0x2) and CPUID(0x4) refactorings:

- `[PATCH v1 00/40] x86: CPUID(0x2) and CPUID(0x4) refactorings`
- `[PATCH v4 00/29] x86: Leaf 0x2 and leaf 0x4 refactorings`

(III) Merging CPUID bit listings to tools/kcpuid

Latest x86-cpuid-db generated CSV files were regularly posted and merged:

- `[PATCH v3 00/20] tools/x86/kcpuid: Update bitfields to x86-cpuid-db v2.3`
- `[PATCH] tools/x86/kcpuid: Update bitfields to x86-cpuid-db v2.4`

(IV) Bug fixes for various CPUID callsites

Individual CPUID call sites:

- `[PATCH v2 0/2] x86/cacheinfo: Fixes for CPUID(0x80000005) and CPUID(0x80000006)`
- `[PATCH v1 0/2] x86: CPUID refactorings: follow-up`

(V) CPUID headers organization

Refactoring CPUID headers into:

- include/asm/cpuid/
  - api.h
  - leaf_types.h // x86-cpuid-db generated file
  - types.h

- [PATCH v1 0/9] x86/cpuid: Headers reorganization

Disentangle circular header dependencies:

- [PATCH v2 0/6] x86: Disentangle <asm/processor.h> dependency on CPUID headers

(VI) Introducing a centralized CPUID model

- [PATCH v1 00/26] x86: Introduce centralized CPUID model
- ...
- [PATCH v5 00/35] x86: Introduce a centralized CPUID data model

(VII) Centralized API

```
/* For leaves with static compile-time, subleaves */
cpuid_subleaf(_cpuinfo, _leaf, _subleaf)
cpuid_leaf(_cpuinfo, _leaf)
cpuid_leaf_regs(_cpuinfo, _leaf)


/* For CPUID leaves with dynamic subleaves */
cpuid_subleaf_index(_cpuinfo, _leaf, _idx)
cpuid_subleaf_index_regs(_cpuinfo, _leaf, _idx)
cpuid_subleaf_count(_cpuinfo, _leaf)
```

(VII) Centralized API: Benefits

- Return CPUID output as typed, auto-generated, C structures
- Return cached, CPUID output. No invocation of same CPUID query
- Return sanitized CPUID data (max leaves, vendor, dependencies, ...)
- A centralized global view of system CPUID data (kernel-enforced feature masking or overrides)

(VIII) Converted Call sites

```
CPUID(0x0), CPUID(0x1), CPUID(0x2), CPUID(0x4),
CPUID(0x6), CPUID(0x7).0, CPUID(0x7).1, CPUID(0xd).1,
CPUID(0x16), CPUID(0x80000000)=>CPUID(0x80000008),
CPUID(0x8000000a), CPUID(0x8000001d),
CPUID(0x8000001f), CPUID(0x80000021),
CPUID(0x80860000)=>CPUID(0x80860006),
CPUID(0xc0000001)
```

Questions / Comments

Thank you for your attention.

darwi@linutronix.de
info@linutronix.de